

```

{
    public static void Test()
    {
        string s1 = "42";
        string s2 = "not 42";
// Int32.Parse throws a System.FormatException if
//the string is not a number. In Java, the equivalent,
//NumberFormatException is a checked exception so we would
//be required to wrap the conversion in a try-catch block.
//In C#, there is no requirement to do so.
        int x1 = Int32.Parse(s1);
        int x2 = Int32.Parse(s2);
    }
}

```

There are also general interfaces that you should be aware of in System.

- `ICloneable` provides a `Clone` method for deep-cloning of objects (vs. shallow or reference copying provided by `Object.MemberwiseClone`).
- `IComparable<T>` provides ordering between objects.
- `IEquatable<T>` provides equality checking between objects.
- `IDisposable` provides deterministic clean-up of resources. We'll revisit this when we talk about IO.

```

public class CloneableCell<T> where T : ICloneable
{
    public T Cell { get; private set; }
    public CloneableCell(T t) { Cell = t; }
    public CloneableCell<T> Clone()
    {
        return new CloneableCell<T>((T) Cell.Clone());
    }
}

```

5.3 The System.Collections namespace

This namespace is broken up into the pre-generic collections found in `System.Collections` and their generic variants in `System.Collections.Generic`. This can be confusing because if you introduce both namespaces into your file (as VS does by default for new code files), you'll see, e.g., `ICollection` and `ICollection<T>`. Just keep in mind which variant you are using, usually the generic variant.